

Отчет о постановке задач о возможности смены инструмента трансляции на ABML

Бояндин Лев, Харьков Александр - НГУ

1 октября 2026 г.

Работа пока что проводится только над уже верифицированными версиями программ (то есть до версии 2 включительно), поскольку сначала необходимо выработать общую методику на уже проверенных функциях.

Поставленные цели для подкоманды:

- Цель 1. Выделить присутствующие конструкции языка C;
- Цель 2. Построить для этих конструкций онтологию на языке ABML;
- Цель 3. Построить аксиоматическую семантику.

Верифицированные реализации алгоритма умножения матриц:

- Версия 0. Наивная реализация.
- Версия 1. С переставленными внутренними циклами.
- Версия 2. С интринсиками.

Версия 0. Наивная реализация

```
1 void gemm_v0(int M, int N, int K, const float* A, const float* B, float* C)
2 {
3     for (int i = 0; i < M; ++i)
4     {
5         for (int j = 0; j < N; ++j)
6         {
7             C[i*N + j] = 0;
8             for (int k = 0; k < K; ++k)
9                 C[i*N + j] += A[i*K + k] * B[k*N + j];
10        }
11    }
12 }
```

Версия 1. С переставленными внутренними циклами

```
1  void gemm_v1(int M, int N, int K, const float* A, const float* B, float* C)
2  {
3      for (int i = 0; i < M; ++i)
4      {
5          float* c = C + i * N;
6          for (int j = 0; j < N; ++j)
7              c[j] = 0;
8          for (int k = 0; k < K; ++k)
9          {
10             const float* b = B + k * N;
11             float a = A[i*K + k];
12             for (int j = 0; j < N; ++j)
13                 c[j] += a * b[j];
14         }
15     }
16 }
```

Версия 2. С интринсиками

```
1 void gemm_v2(int M, int N, int K, const float* A, const float* B, float* C)
2 {
3     for (int i = 0; i < M; ++i)
4     {
5         float* c = C + i * N;
6         for (int j = 0; j < N; j += 8)
7             _mm256_storeu_ps(c + j + 0, _mm256_setzero_ps());
8         for (int k = 0; k < K; ++k)
9         {
10             const float* b = B + k * N;
11             __m256 a = _mm256_set1_ps(A[i*K + k]);
12             for (int j = 0; j < N; j += 16)
13             {
14                 _mm256_storeu_ps(c + j + 0, _mm256_fmadd_ps(a,
15                     _mm256_loadu_ps(b + j + 0), _mm256_loadu_ps(c + j + 0)));
16                 _mm256_storeu_ps(c + j + 8, _mm256_fmadd_ps(a,
17                     _mm256_loadu_ps(b + j + 8), _mm256_loadu_ps(c + j + 8)));
18             }
19         }
20     }
21 }
```

Спецификация __mm256_setzero_ps

```
1  /*@
2     assigns \nothing;
3     ensures  $\forall$  integer  $i$ ;  $0 \leq i \leq 7 \Rightarrow$  \result.e[i] == 0.0;
4  */
5  __mm256 __mm256_setzero_ps(void);
```

Спецификация __mm256_set1_ps

```
1  /*@
2      assigns \nothing;
3      ensures  $\forall$  integer  $i$ ;  $0 \leq i \leq 7 \Rightarrow$  \result.e[i] == v;
4  */
5  __mm256 __mm256_set1_ps(float v);
```

Спецификация `_mm256_loadu_ps`

```
1  /*@
2      requires \valid_read(a + (0 .. 7));
3      assigns \nothing;
4      ensures  $\forall$  integer i;  $0 \leq i \leq 7 \Rightarrow$  \result.e[i] == a[i];
5  */
6  __mm256 _mm256_loadu_ps(const float* a);
```

Спецификация `_mm256_storeu_ps`

```
1  /*@
2      requires \valid(mem_addr + (0 .. 7));
3      assigns mem_addr[0 .. 7];
4      ensures  $\forall$  integer i;  $0 \leq i \leq 7 \Rightarrow$  mem_addr[i] == a.e[i];
5  */
6  void _mm256_storeu_ps(float* mem_addr, __mm256 a);
```

Спецификация __mm256_fmadd_ps

```
1  /*@
2      assigns \nothing;
3      ensures  $\forall$  integer  $i$ ;  $0 \leq i < 8$ 
3       $\Rightarrow$  \result.e[i] == a.e[i] * b.e[i] + c.e[i];
4  */
5  __mm256 __mm256_fmadd_ps(__mm256 a, __mm256 b, __mm256 c);
```

Что такое ABML?

Предметно-ориентированный язык программирования ABML (Attribute-Based Modeling Language) - это лексическое расширение Common Lisp.

```
1  for (int i = 0; i < M; ++i) {  
2      // Какое-то тело цикла  
3  }
```

ABML: пример онтологии

```
1  (typedef "type" (uniont "int type"))
2  (cot "int type")
3
4  (typedef "variable name" string)
5  (mot "variable acces" :at 1 "variable name")
6  (mot "cell" :at "type" "type" :at "value" "C value")
7
8  (typedef "expression" (uniont "C value" "variable acces" "1<2"))
9  (typedef "C value" (uniont bool int real))
10 (mot "1<2" :at 1 "C value" :at 2 "C value")
11
12 (typedef "statement" (uniont "++1" "for stmt"))
13 (cot "++1" :at 1 "variable name")
14 (mot "for stmt" :at "init" "statement" :at "condition" "expression"
15      :at "post" "statement" :at "body" "block")
16
17 (mot "block" :at "sts" (listt "statement"))
```

Визуализация умножения матриц

